COLLEGE ENGINEERING PROJECT · PROJECT 23 OF 55

AI-Based Meeting Summarization and Action-Item Extraction Using Natural Language Processing

Domain: Speech & Natural Language Processing | **Module:** project_23_meeting_summarization

PROJECT ABSTRACT

Meetings are an important part of academic, business, and organizational activities, where discussions often contain decisions, responsibilities, deadlines, and important follow-up tasks. However, manually recording and reviewing lengthy meeting conversations can be time-consuming, and important action items may be overlooked. The proposed AI-Based Meeting Summarization and Action-Item Extraction System Using Natural Language Processing (NLP) aims to automatically convert meeting conversations into concise and meaningful information that can be easily understood and followed. The proposed system is implemented using Python, Natural Language Processing techniques, and machine learning/deep learning models to process meeting transcripts or speech-to-text content. The system analyzes the conversation, identifies important discussion points, removes unnecessary or repetitive information, and generates a concise summary of the meeting. Instead of simply shortening the entire conversation, the system focuses on extracting information that is useful for understanding the actual outcome of the meeting. The major uniqueness of the proposed system is its action-oriented summarization approach. Along with generating a general meeting summary, the system specifically identifies statements that represent tasks, decisions, responsibilities, and deadlines. For example, when a participant is assigned a particular task during the discussion, the system can recognize the task and associate it with the responsible person and expected completion date. This converts an unstructured conversation into a structured set of actionable information. Another important feature is the ability to distinguish between general discussion and final decisions. Meetings often contain multiple suggestions and opinions before reaching a conclusion. The proposed system attempts to identify the final decisions and important outcomes rather than treating every statement as equally important. This helps users quickly understand what was actually decided and what actions need to be taken after the meeting. The extracted information can be presented as a structured meeting report containing the meeting summary, key discussion points, decisions, action items, responsible participants, and deadlines. Such a format reduces the effort required to prepare meeting minutes manually and provides participants with a clear reference for future follow-up. The system can also be extended to process meetings from different domains, including project management, education, software development, team discussions, and organizational meetings. The proposed system can further support conversational or real-time meeting environments by processing speech converted into text and continuously identifying important information. With suitable integration, extracted action items can also be connected to task-management or notification systems, helping users remember pending responsibilities and

> deadlines. Overall, the project demonstrates how NLP and AI can transform lengthy and unstructured meeting conversations into concise, organized, and actionable knowledge. By identifying not only what was discussed but also what was decided, who is responsible, and what needs to be completed, the proposed system provides a practical solution for improving meeting productivity, accountability, and follow-up efficiency.

📌 Project Metadata & Specifications

Project Title	AI-Based Meeting Summarization and Action-Item Extraction Using Natural Language Processing
Domain / Specialization	Speech & Natural Language Processing
Source Directory	`project_23_meeting_summarization`
Backend REST API	`http://localhost:5023` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 23 — PHASE 1

Phase 1 — Requirements, Data and Prototype
The first phase will convert the project concept into a measurable prototype for NLP/language or intelligent information processing. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.
1. Problem Definition and Scope
Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.
2. Data Preparation
Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.
3. Baseline Model and Experiment Setup
Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so

that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

- Define text cleaning, tokenization/embedding and document-length handling rules.
- Create representative examples for each output class or information field.
- Track confidence and ambiguous cases for human review rather than forcing uncertain outputs.

PROJECT 23 — PHASE 2

Phase 2 — Optimization, Integration and Validation

The second phase will turn the Phase 1 prototype into a validated project for NLP/language or intelligent information processing. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.

1. Model Improvement

Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.

2. End-to-End Integration

Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.

3. Testing and Evaluation

Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.

4. Demonstration and Final Evidence

Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.

Phase 2 Deliverables

Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.

Project-Specific Phase 2 Validation

- Compare the final model with the Phase 1 baseline using the same held-out evaluation set.

• Document representative success cases and failure cases instead of reporting only aggregate accuracy.
• Prepare a final demonstration script covering input, processing, output, validation and limitations.

Final Expected Outcome

A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel /college project/project_23_meeting_summarization/backend" # Install project dependencies pip install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5023`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_23_meeting_summarization/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.